

JavaScript

ES6+ Features

Beginner to Expert • Interview Prep • Code Examples

ES6 (2015) → `let/const`, `arrows`, `classes`, `promises`, `modules`

ES7 (2016) → `Array.includes()`, `exponentiation **`

ES8 (2017) → `async/await`, `Object.entries()/values()`

ES9 (2018) → `Rest/Spread` for objects, `Promise.finally()`

ES10 (2019) → `flat()`, `flatMap()`, `optional catch binding`

ES11 (2020) → `optional chaining ?.`, `nullish coalescing ??`

ES12 (2021) → `logical assignment ||= &&= ??=`, `Promise.any()`

ES13 (2022) → `Array.at()`, `Object.hasOwn()`, `private fields #`

ES14 (2023) → `Array.toSorted()`, `toReversed()`, `findLast()`

let, const vs var

```
// var – function scoped, hoisted, re-declarable

var x = 1;

var x = 2; // no error – dangerous!


// let – block scoped, NOT re-declarable

let a = 1;

// let a = 2; SyntaxError!


// const – block scoped, must be initialized

const PI = 3.14;

// PI = 3; TypeError – can't reassign!


// Gotcha – const object properties CAN be mutated

const user = { name: 'Akshay' };

user.name = 'Rahul'; // OK – reference is const, not content

// user = {}; TypeError – can't reassign reference
```

Keyword	Scope	Re-declare	Re-assign
var	Function	Yes	Yes
let	Block	No	Yes
const	Block	No	No

Arrow Functions & `this` Binding

```
// Regular function

function add(a, b) { return a + b; }


// Arrow – same thing, shorter

const add = (a, b) => a + b;

const double = n => n * 2; // single param, no parens

const greet = () => 'Hello!'; // no params


// Multi-line needs return

const multiply = (a, b) => {
```

```
const result = a * b;

return result;

};
```

Arrow functions DON'T have their own `this`` — they inherit `this`` from where they were DEFINED, not called. Never use arrow functions as object methods.

```
const obj = {

  name: 'Akshay',

  regularFn: function() {

    console.log(this.name); // 'Akshay' — this = obj

  },

  arrowFn: () => {

    console.log(this.name); // undefined — this = outer scope!

  }

};
```

Object & Array Destructuring

Beginner

```
// Object Destructuring

const user = { name: 'Akshay', age: 25, city: 'Pune' };

const { name, age } = user; // basic

const { name: userName } = user; // rename

const { country = 'India' } = user; // default value


// Nested destructuring

const data = { user: { address: { city: 'Pune' } } };

const { user: { address: { city } } } = data;

console.log(city); // 'Pune'


// Array Destructuring

const [first, second, , fourth] = [10, 20, 30, 40];

console.log(first, fourth); // 10, 40 (skipped 30)


// Swap variables

let a = 1, b = 2;

[a, b] = [b, a];

console.log(a, b); // 2, 1
```

Spread & Rest Operators (...)

Beginner

```
// SPREAD – expands items OUT of array/object

const arr1 = [1, 2, 3];

const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]

const merged = { ...obj1, ...obj2 }; // merge objects

console.log(Math.max(...nums)); // pass as args


// REST – collects many items INTO one array

function sum(...numbers) {

  return numbers.reduce((a, b) => a + b, 0);

}

console.log(sum(1, 2, 3, 4)); // 10
```

```
const [head, ...tail] = [1, 2, 3, 4];
```

```
// head = 1, tail = [2, 3, 4]
```

Spread = 1 array → many items (expanding outward)

Rest = many items → 1 array (collecting inward)

Optional Chaining ?. and Nullish Coalescing ??

```
// Optional chaining – safe navigation

const user = { name: 'Akshay', address: { city: 'Pune' } };

// Old way – verbose

const city = user && user.address && user.address.city;

// ES2020 way – clean

const city = user?.address?.city; // 'Pune'

const phone = user?.contact?.phone; // undefined, no crash!

const first = user?.hobbies?.[0]; // works on arrays too

const name = user?.getName?.(); // works on methods too
```

```
// Nullish Coalescing ??

// Returns right side ONLY if left is null or undefined

const count = 0;

console.log(count || 10); // 10 – 0 is falsy, wrong!

console.log(count ?? 10); // 0 – 0 is not null/undefined!

const name = '';

console.log(name || 'Guest'); // 'Guest' – empty string falsy

console.log(name ?? 'Guest'); // '' – empty string OK!

// Combined – the power combo

const port = config?.server?.port ?? 3000;
```

Promises & Promise Combinators

Intermediate →
Expert

```
// Promise states: pending → fulfilled OR rejected

const fetchUser = (id) => new Promise((resolve, reject) => {

  if (id > 0) resolve({ id, name: 'Akshay' });

  else reject(new Error('Invalid ID'));

});

// Chaining

fetchUser(1)

.then(user => user.name) // transform

.then(name => name.toUpperCase()) // chain further

.catch(err => console.error(err)) // catch ANY error

.finally(() => console.log('Done')); // always runs
```

Method	Behaviour	Use When
Promise.all()	ALL resolve, fails fast on first rejection	Need all results
Promise.allSettled()	Waits for ALL, never rejects	Want all results even if some fail
Promise.race()	First to settle wins (resolve or reject)	Timeout patterns
Promise.any()	First to RESOLVE wins, ignores rejections	First success matters

SECTION 07 — ASYNC / AWAIT

async/await — Under the Hood

Intermediate →
Expert

async/await is syntactic sugar over Promises. Every async function always returns a Promise, even if you return a plain value.

```
async function getUser(id) {

  try {

    const response = await fetch(`/api/users/${id}`);

    const data = await response.json();

    return data; // resolves the returned Promise

  } catch (err) {

    throw err; // rejects the returned Promise

  }

}
```


Symbols — Unique Keys & Well-Known Symbols

Expert

```
const s1 = Symbol('id');

const s2 = Symbol('id');

console.log(s1 === s2); // false — always unique!

// Hidden object keys (not in for..in / Object.keys)

const _id = Symbol('id');

const user = { name: 'Akshay', [_id]: 101 };

console.log(Object.keys(user)); // ['name'] — Symbol hidden!

console.log(user[_id]); // 101

// Well-known Symbols — customise JS behaviour

class MyArray {

  [Symbol.iterator]() { // makes object iterable!

    let i = 0;

    const data = [10, 20, 30];

    return { next() { return i < data.length

      ? { value: data[i++], done: false }

      : { done: true }; } }

  }

}

for (const val of new MyArray()) console.log(val); // 10,20,30
```

Logical Assignment Operators

Intermediate

```
let a = null;

a ||= 'default'; // assign only if a is FALSY

console.log(a); // 'default'

let b = 'Akshay';

b &&= b.toUpperCase(); // assign only if b is TRUTHY

console.log(b); // 'AKSHAY'

let c = 0;
```

```
c ??= 42; // assign only if c is null/undefined

console.log(c); // 0 – 0 is not null/undefined!
```

SECTION 10 — TAGGED TEMPLATE LITERALS

Tagged Template Literals

Expert

```
// Tags are functions that process template literals

function highlight(strings, ...values) {
  return strings.reduce((result, str, i) => {
    return result + str + (values[i] ? `${values[i]}` : '');
  }, '');
}

const name = 'Akshay';
const skill = 'JavaScript';

const out = highlight`Hello ${name}, you know ${skill}!`;

// 'Hello Akshay, you know JavaScript!'

// Real use: SQL injection protection

const id = userInput;

const query = sql`SELECT * FROM users WHERE id = ${id}`;

// Library sanitizes 'id' automatically
```

Top Interview Questions — ES6+ Features

Q: What is the difference between ?? and ||?

→ || treats all falsy values (0, "", false, null, undefined) as missing. ?? only treats null and undefined as missing. So `0 ?? 10 = 0` but `0 || 10 = 10`.

Q: Why shouldn't arrow functions be used as object methods?

→ Arrow functions don't have their own `this`. As object methods, `this` points to the outer scope (window/undefined), not the object.

Q: What is the difference between Promise.all and Promise.allSettled?

→ Promise.all fails immediately if any promise rejects. Promise.allSettled waits for ALL promises and returns results for each — fulfilled or rejected.

Q: What does ?. short circuit mean?

→ If the left side of ?. is null or undefined, the entire chain returns undefined immediately without evaluating the rest.

Q: What is the difference between spread and rest?

→ Spread expands an array/object into individual items. Rest collects individual arguments into an array. Same syntax (...), opposite direction.

Q: What are well-known Symbols?

→ Built-in Symbols like Symbol.iterator, Symbol.toPrimitive that let you customise how JavaScript handles your objects in native operations like for...of, type coercion etc.

Quick Reference Cheatsheet

let / const → Block scoped, no re-declare. Use const by default.

Arrow fn → No own `this`, perfect for callbacks, not methods.

?. → Safe navigation, returns undefined instead of crashing.

?? → Fallback only for null/undefined, not all falsy values.

Promise.all → Parallel requests, fails fast. allSettled never fails.

async/await → Syntactic sugar over Promises. Use Promise.all to parallelize.